

MLE5004: Arhitectura sistemelor de calcul

Examen scris, Timp de lucru: 1h 30'

21 Ianuarie 2026

1. Definiți, clasificați și dați exemple comentate pentru următoarele noțiuni:

- a. Directivă
- b. Moduri de specificare a operanzilor
- c. Operatori
- d. Conceptul de **call code** în contextul general al unui limbaj de programare. Studiu de caz ASM + ASM și ASM + C: cine este responsabil pentru generarea **call code**-ului și când este generat acesta?

2. Dându-se următoarea secvență ASM:

```
x db 1, 2, 3
y dd -3
z dw 65533

mov ax, word [x+1]
neg al
not ah
mov bx, word [x+2]
add bh, bl
adc bl, bh
cbw
sub bx, ax
mov edi, [y+2]
inc edi
mov cx, word [x]
xor cx, di
xchg ch, cl
dec ch
movzx ecx, cx
again:
    shr cl, 1
    inc bx
loop again
```

Cerință: Prezentați structura memoriei pentru segmentul de date (variabilele). Explicați și justificați efectul fiecărei linii sursă (dacă o considerați corectă sintactic; dacă nu, justificați și ignorați-o). Arătați valorile registrelor implicate în baza 16 (interpretare cu semn și fără semn). De câte ori se execută bucla **again**?

--

3. (NU SUNT SIGUR CA E ENUNTUL CORECT) Dându-se segmentul de date:

```
a dw "2", "4", "6"
len equ ($-a)/2
format db "%u", 0
```

Cerință: Scrieți o secvență de instrucțiuni care să afișeze valoarea 256 pe ecran folosind definițiile de mai sus.

4. Completați liniile A, B, C, D, E în secvența de mai jos astfel încât cifrele binare ale numărului din variabila **a** să fie stocate în șirul **digits**:

```
segment data use32 class=data
    a dd ... ; a este initializat aici
    digits times 32 db 0
segment code use32 class=code
start:
    _____ ; A
    mov ecx, 32
    mov eax, [a]
et_1:
    _____ ; B
    _____ ; C
    mov byte [digits+ebx], '0'
    jmp et_3
et_2:
    mov byte [digits+ebx], '1'
et_3:
    _____ ; D
    _____ ; E
    loop et_1
```

5. Dându-se următoarele două secvențe ASM:

Secvența 1:

```
mov ah, 0bch
mov al, 0deh
add ah, al
```

Secvența 2:

```
mov dh, 62h
mov ch, 200
sub dh, ch
```

Cerință: Care este rezultatul? Detaliați efectul fiecărei linii, dând interpretarea în bazele 10 și 16 (cu semn și fără semn) pentru fiecare valoare. Specificați dacă apare conceptul de **overflow** (depășire), precizând cauza și efectul.

6. Două șiruri de cuvinte de aceeași lungime N sunt date în segmentul de date. Folosind doar instrucțiuni pe șiruri (precum și elemente specifice modului lor de acțiune), scrieți o secvență de cod care, pornind de la șirul sursă, caută prima apariție a primului octet din `sir1` în `sir2`, parcurgând de la sfârșitul lui `sir2` spre început.

```
segment data use32 class=data
    sir1 dw 3, 4, -1, 6, 8, ...
    sir2 dw 5, 1, 7, 9, -3, ...
    lung EQU $-sir2
segment code use32 class=code
start:
    mov ESI, sir1
    mov EDI, sir2+lung-2
```

7. La începutul execuției secvenței de mai jos, dublucuvintele cu valorile 0, -1, -2, -4, -8 sunt puse pe stivă în această ordine. Care sunt elementele din șirul `sir` care vor fi accesate și care este ordinea lor exactă?

```
segment data use32 class=data
    sir db '1234567890'
segment code use32 class=code
start:
    ; cele 5 numere sunt deja pe stiva
    mov ebx, sir
    mov ecx, 10
eti:
    pop ax
    neg ax
    xlat
    loop eti
```

8. Pornind de la un șir de cuvinte de lungime `len_a`, scrieți o secvență de cod care creează două șiruri de octeți **B1** și **B2**, astfel încât **B1** să conțină doar octeții superiori (cei mai semnificativi) ai cuvintelor din șirul inițial, iar **B2** să conțină doar octeții inferiori (cei mai puțin semnificativi).

Exemplu: a: 1234h, 1a2bh -> B1: 12h, 1ah și B2: 34h, 2bh

```
segment data use32 class=data
```

```
segment code use32 class=code  
start:
```